

AG3 - Actividad Guiada 3

Nombre: Pedro Borregón González

Link: <https://colab.research.google.com/drive/11D5ppN3lUwM932sG9H1dgYdSxi4GGrwR?usp=sharing>

Github: https://github.com/pborregong/AlgoritmosOptimizacionFinal/blob/main/Algoritmos_AG3_Pedro_Borregon_Gonzalez.ipynb

✓ Carga de librerías

```
!pip install requests      #Hacer llamadas http a paginas de la red
!pip install tsplib95      #Modulo para las instancias del problema del TSP

Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests) (2026.7.22)
Collecting tsplib95
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.13/dist-packages (from tsplib95) (8.5.0)
Collecting Deprecated~1.2.9 (from tsplib95)
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl.metadata (5.7 kB)
Collecting networkx~2.1 (from tsplib95)
  Downloading networkx-2.8.8-py3-none-any.whl.metadata (5.1 kB)
Collecting tabulate~0.8.7 (from tsplib95)
  Downloading tabulate-0.8.10-py3-none-any.whl.metadata (25 kB)
Collecting wrapt<2,>=1.10 (from Deprecated~1.2.9->tsplib95)
  Downloading wrapt-1.17.3-cp313-cp313-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl.metadata (6.4 kB)
Downloading tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
Downloading Deprecated-1.2.18-py2.py3-none-any.whl (10.0 kB)
Downloading networkx-2.8.8-py3-none-any.whl (2.0 MB)
----- 2.0/2.0 MB 26.6 MB/s eta 0:00:00
Downloading tabulate-0.8.10-py3-none-any.whl (29 kB)
Downloading wrapt-1.17.3-cp313-cp313-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl (88 kB)
----- 88.1/88.1 kB 7.8 MB/s eta 0:00:00
Installing collected packages: wrapt, tabulate, networkx, Deprecated, tsplib95
  Attempting uninstall: wrapt
    Found existing installation: wrapt 2.4.0
    Uninstalling wrapt-2.4.0:
      Successfully uninstalled wrapt-2.4.0
  Attempting uninstall: tabulate
    Found existing installation: tabulate 0.9.0
    Uninstalling tabulate-0.9.0:
      Successfully uninstalled tabulate-0.9.0
  Attempting uninstall: networkx
    Found existing installation: networkx 3.6.1
    Uninstalling networkx-3.6.1:
      Successfully uninstalled networkx-3.6.1
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of
mapclassify 2.10.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
bigframes 2.48.0 requires tabulate>=0.9, but you have tabulate 0.8.10 which is incompatible.
spopt 0.7.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
scikit-image 0.25.2 requires networkx>=3.0, but you have networkx 2.8.8 which is incompatible.
momepy 0.11.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
Successfully installed Deprecated-1.2.18 networkx-2.8.8 tabulate-0.8.10 tsplib95-0.7.1 wrapt-1.17.3
```

✓ Carga de los datos del problema

```
import urllib.request #Hacer llamadas http a paginas de la red
import tsplib95        #Modulo para las instancias del problema del TSP
import math            #Modulo de funciones matematicas. Se usa para exp
import random          #Para generar valores aleatorios

#http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
#Documentacion :
# http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
# https://tsplib95.readthedocs.io/en/stable/pages/usage.html
# https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
# https://pypi.org/project/tsplib95/

#Descargamos el fichero de datos(Matriz de distancias)
file = "swiss42.tsp" ;
# urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/swiss42.tsp.gz", file + '.gz')
# !gzip -d swiss42.tsp.gz      #Descomprimir el fichero de datos

#URL alternativa
# file = "swiss42.tsp" ;
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/heads/master/swiss42.tsp", file )

#Coordendas 51-city problem (Christofides/Eilon)
```



```
#Comprobar los 10 primeros Nodos
print(Nodos[:10])
```

```
#Comprobar las 10 primeras Aristas
print(Aristas[:10])
```

```
swiss42
42
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9)]
```

```
NOMBRE: swiss42
TIPO: TSP
COMENTARIO: 42 Staedte Schweiz (Fricker)
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 72 78 82 159 122 131 206 112 57 28 43 70 1
15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 5
30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18
23 23 11 0 11 48 26 54 70 94 69 75 75 84 84 89 92 89 54 78 99 141 111 109 89 89 11 11 11 54
32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35
55 40 57 48 40 0 23 55 96 123 78 75 36 36 66 66 63 95 34 34 137 174 156 129 224 90 15 59 75
33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54
37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 40 40 87 87 94 158 158 163 242 135 65 6
92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65
114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 11
92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 6
110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 9
96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79
90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95
74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 9
```

```
#Probamos algunas funciones del objeto problem
```

```
#Distancia entre nodos
problem.get_weight(0, 1)
```

```
#Todas las funciones
#Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
```

```
#dir(problem)
```

```
15
```

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import MDS # Multidimensional Scaling o Escalado Multidimensional

def plot_tsp_solution(distance_matrix, tsp_solution):
    """
    Dibuja el grafo de un TSP con las posiciones calculadas mediante MDS y muestra
    solo las aristas correspondientes a la solución del TSP.

    :param distance_matrix: np.ndarray, matriz de distancias entre nodos
    :param tsp_solution: list, lista de nodos en el orden de la solución del TSP
    """
    # Crear el grafo completo
    G = nx.Graph()
    num_nodes = len(distance_matrix)
    for i in range(num_nodes):
        for j in range(i+1, num_nodes):
            G.add_edge(i, j, weight=distance_matrix[i][j])

    # Usar MDS para calcular posiciones de los nodos
    mds = MDS(n_components=2, dissimilarity="precomputed", random_state=42)
    positions = mds.fit_transform(distance_matrix)

    # Convertir las posiciones en un diccionario para networkx
    pos = {i: positions[i] for i in range(num_nodes)}

    # Crear un subgrafo con las aristas del camino TSP
    TSP_G = nx.Graph()
    for i in range(len(tsp_solution)-1):
        u = tsp_solution[i]
        v = tsp_solution[i+1]
        TSP_G.add_edge(u, v, weight=distance_matrix[u][v])

    # Dibujar el grafo
    plt.figure(figsize=(8, 6))

    # Dibujar nodos
    nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=500)

    # Dibujar las aristas del camino TSP
    nx.draw_networkx_edges(TSP_G, pos, edge_color='red', width=2)

    # Añadir etiquetas a los nodos y pesos de las aristas
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
    edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
```

```

edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
...nx.draw_networkx_edge_labels(TSP_G, pos, edge_labels=edge_labels, font_size=8)

...plt.title("Grafo solucion TSP")
...plt.show()

```

Funcionas basicas

```

#Funcionas basicas
#####

#Se genera una solucion aleatoria con comienzo en en el nodo 0
def crear_solucion(Nodos):
    solucion = [Nodos[0]]
    for n in Nodos[1:]:
        solucion = solucion + [random.choice(list(set(Nodos) - set({Nodos[0]}) - set(solucion)))]
    return solucion

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
    return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
    distancia_total = 0
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i] ,solucion[i+1] , problem)
    return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0], problem)

sol_temporal = crear_solucion(Nodos)

distancia_total(sol_temporal, problem), sol_temporal

```

```

(4767,
 [0,
  20,
  2,
  14,
  8,
  3,
  32,
  15,
  13,
  24,
  12,
  1,
  41,
  21,
  5,
  31,
  33,
  11,
  17,
  38,
  25,
  40,
  35,
  6,
  19,
  16,
  39,
  22,
  9,
  28,
  23,
  4,
  27,
  29,
  37,
  10,
  30,
  26,
  18,
  36,
  34,
  7])

```

BUSQUEDA ALEATORIA

```

#####
# BUSQUEDA ALEATORIA
#####

def busqueda_aleatoria(problem, N):
    #N es el numero de iteraciones
    Nodos = list(problem.get_nodes())

```

```

mejor_solucion = []
#mejor_distancia = 10e100                                #Inicializamos con un valor alto
mejor_distancia = float('inf')                            #Inicializamos con un valor alto

for i in range(N):
    solucion = crear_solucion(Nodos)                      #Criterio de parada: repetir N veces pero podemos incluir otros
    distancia = distancia_total(solucion, problem)         #Genera una solucion aleatoria
                                                         #Calcula el valor objetivo(distancia total)

    if distancia < mejor_distancia:                       #Compara con la mejor obtenida hasta ahora
        mejor_solucion = solucion
        mejor_distancia = distancia

print("Mejor solución:" , mejor_solucion)
print("Distancia      : " , mejor_distancia)
return mejor_solucion

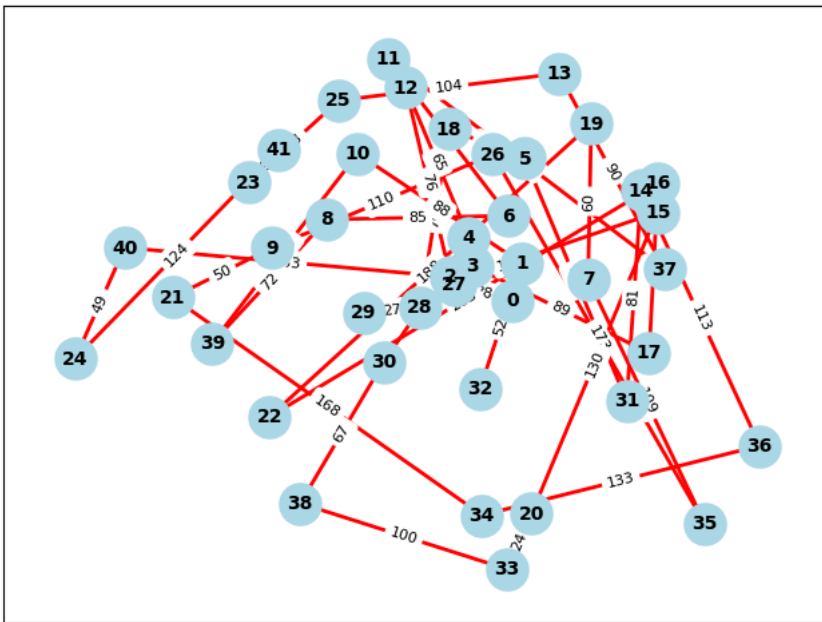
#Busqueda aleatoria con 10000 iteraciones
solucion = busqueda_aleatoria(problem, 10000)

plot_tsp_solucion(problem.edge_weights, solucion)

```

Mejor solución: [0, 27, 3, 17, 16, 22, 19, 7, 35, 26, 9, 21, 34, 36, 14, 31, 5, 18, 28, 29, 15, 20, 33, 38, 30, 4, 12, 2, 40, 24, 41, 23,
Distancia : 3749

Grafo solucion TSP



BUSQUEDA LOCAL

```

#####
# BUSQUEDA LOCAL
#####
def genera_vecina(solucion):
    #Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nodos se generan (N-1)x(N-2)/2 soluciones
    #Se puede modificar para aplicar otros generadores distintos que 2-opt
    #print(solucion)
    mejor_solucion = []
    mejor_distancia = 10e100
    for i in range(1,len(solucion)-1):                    #Recorremos todos los nodos en bucle doble para evaluar todos los intercambios 2-opt
        for j in range(i+1, len(solucion)):

            #Se genera una nueva solución intercambiando los dos nodos i,j:
            # (usamos el operador + que para listas en python las concatena) : ej.: [1,2] + [3] = [1,2,3]
            vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]

            #Se evalua la nueva solución ...
            distancia_vecina = distancia_total(vecina, problem)

            #... para guardarla si mejora las anteriores
            if distancia_vecina <= mejor_distancia:
                mejor_distancia = distancia_vecina
                mejor_solucion = vecina
    return mejor_solucion

```

```

#solucion = [1, 47, 13, 41, 40, 19, 42, 44, 37, 5, 22, 28, 3, 2, 29, 21, 50, 34, 30, 9, 16, 11, 38, 49, 10, 39, 33, 45, 15, 24, 43, 26, 3
print("Distancia Solucion Inicial:" , distancia_total(solucion, problem))

```

Distancia Solucion Incial: 3663
Distancia Mejor Solucion Local: 3431

En la iteración 33, la mejor solución encontrada es: [0, 28, 30, 29, 8, 10, 41, 23, 9, 21, 40, 24, 39, 22, 38, 32, 34, 33, 20, 35, 36, 1
Distancia : 1673

```
#####
# SIMULATED ANNEALING
#####

#Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
#Mejorable eligiendo otra forma de elegir una vecina.
def genera_vecina_aleatorio(solucion):
```

```

#Se eligen dos nodos aleatoriamente
i,j = sorted(random.sample( range(1,len(solucion)) , 2))

#Devuelve una nueva solución pero intercambiando los dos nodos elegidos al azar
return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]

#Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T,d):
    if random.random() < math.exp( -1*d / T) :
        return True
    else:
        return False

#Funcion de descenso de temperatura
def bajar_temperatura(T):
    return T*0.99

```

```

def recocido_simulado(problem, TEMPERATURA ):
    #problem = datos del problema
    #T = Temperatura

    solucion_referencia = crear_solucion(Nodos)
    distancia_referencia = distancia_total(solucion_referencia, problem)

    mejor_solucion = []          #x* del pseudocódigo
    mejor_distancia = 10e100     #F* del pseudocódigo

    N=0
    while TEMPERATURA > .0001:
        N+=1
        #Genera una solución vecina
        vecina =genera_vecina_aleatorio(solucion_referencia)

        #Calcula su valor(distancia)
        distancia_vecina = distancia_total(vecina, problem)

        #Si es la mejor solución de todas se guarda(siempre!!!)
        if distancia_vecina < mejor_distancia:
            mejor_solucion = vecina
            mejor_distancia = distancia_vecina

        #Si la nueva vecina es mejor se cambia
        #Si es peor se cambia según una probabilidad que depende de T y delta(distancia_referencia - distancia_vecina)
        if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, abs(distancia_referencia - distancia_vecina) ) :
            #solucion_referencia = copy.deepcopy(vecina)
            solucion_referencia = vecina
            distancia_referencia = distancia_vecina

        #Bajamos la temperatura
        TEMPERATURA = bajar_temperatura(TEMPERATURA)

    print("La mejor solución encontrada es " , end="")
    print(mejor_solucion)
    print("con una distancia total de " , end="")
    print(mejor_distancia)
    return mejor_solucion

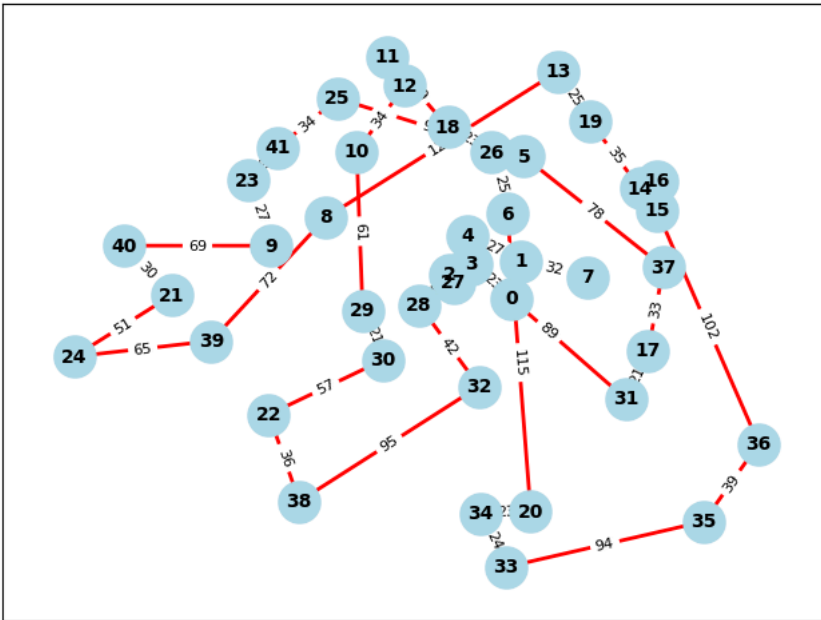
sol = recocido_simulado(problem, 10000000)

plot_tsp_solution(problem.edge_weights, sol)

```

La mejor solución encontrada es [0, 3, 31, 17, 37, 5, 25, 41, 23, 9, 40, 21, 24, 39, 8, 13, 19, 14, 16, 15, 36, 35, 33, 34, 20, 6, 26, 18, con una distancia total de 1883

Grafo solución TSP



Actividades extra

Mejora de algoritmo de búsqueda local

- Multiarranque